1.5 Systems Software

OCR GCSE Computer Science (J277) — Paper 1

> **Systems software** manages the computer's hardware resources and provides a platform for application software to run. It acts as an intermediary between the user, application software, and hardware. Without systems software, applications would have to communicate directly with hardware — every program would need to know the specific details of every possible printer, graphics card, and storage device. The two main types are the **operating system** and **utility software**.

Systems software vs application software: Systems software manages and maintains the computer itself (OS, drivers, anti-malware, disk utilities). Application software performs tasks for the user (word processor, web browser, games, email client). Systems software runs in the background; application software runs in the foreground for the user to interact with.

1.5.1 Operating System Functions

The **operating system (OS)** is the most important piece of systems software. It is loaded into RAM when the computer starts (the boot process) and runs continuously until shutdown. Without it, the user would have to interact directly with the hardware using machine code. The OS provides an abstraction layer — it hides the complexity of the hardware and presents a simple, consistent interface to both users and applications.

The OS manages **six key functions**. You must be able to explain each one in detail:

1. User Interface

The user interface provides a way for the user to interact with the computer, issue commands, and receive feedback.

© RAG Learning · Not for redistribution

Interface Type	Description	Advantages	Disadvantages
GUI (Graphical User Interface)	Windows, Icons, Menus, Pointer (WIMP). Users interact by clicking, dragging, and using visual elements.	Intuitive and easy to learn; visual feedback; good for casual users; supports multitasking with multiple windows	Uses more system resources (RAM, CPU, GPU); slower for repetitive tasks; limited control compared to CLI
CLI (Command Line Interface)	Users type text commands (e.g., `dir`, `ls`, `cd`, `mkdir`). Output is displayed as text.	Very powerful and flexible; faster for experienced users; uses fewer resources; can be scripted/automated; precise control over the system	Steep learning curve; must memorise commands; easy to make mistakes; not visual
Touch / Gesture	Users interact via finger gestures (tap, swipe, pinch, drag) on a touchscreen.	Very intuitive; no keyboard/mouse needed; natural for mobile devices	Less precise than mouse; limited screen size on mobile; on-screen keyboard takes up space; finger obscures content
Voice	Users speak commands processed by speech recognition (e.g., Siri, Alexa, Cortana).	Hands-free; accessible for users with physical disabilities	Accuracy varies; privacy concerns (always listening); not suitable for noisy environments; limited command set

2. Memory Management

The OS is responsible for allocating and managing RAM to ensure all running programs have the memory they need without interfering with each other.

- **Allocation:** When a program opens, the OS allocates a block of RAM for its code, data, and stack. When the program closes, that RAM is freed (deallocated) and made available for other programs.

- **Memory protection:** The OS prevents one program from accessing another program's memory space. Without this, a bug in one program could corrupt data in another, causing crashes or security vulnerabilities. Each process has its own protected memory space.
- **Virtual memory:** If RAM is full, the OS uses a section of the hard drive/SSD as overflow (the page file or swap space). It divides memory into fixed-size **pages** and swaps them between RAM and disk as needed. This allows programs that need more memory than physically available to still run, but at a significant performance cost because disk access is much slower than RAM.
- **Paging:** The OS divides both RAM and virtual memory into pages of equal size (typically 4 KB). A page table maps virtual addresses (what the program thinks) to physical addresses (where the data actually is in RAM or on disk). When a program accesses a page that is on disk, a **page fault** occurs — the OS must load that page into RAM, potentially evicting another page.

Why excessive virtual memory use (thrashing) is bad: If the OS is constantly swapping pages between RAM and disk because too many programs are running, the system spends more time managing memory than doing useful work. The hard drive thrashes (constant read/write activity) and the computer becomes extremely slow. The solution is to add more physical RAM or close some programs.

3. Process (Multitasking) Management

Modern computers appear to run multiple programs simultaneously, but a single CPU core can only execute one instruction at a time. The OS creates this illusion through **time-slicing** and **scheduling**.

- Each running program is a **process** with its own allocated resources (memory, CPU time, file handles).
- The OS uses a **scheduler** to decide which process gets CPU time and for how long.
- **Time-slicing**: The scheduler gives each process a small time slice (typically a few milliseconds) of CPU time, then switches to the next process. This happens thousands of times per second, so the user perceives all programs running simultaneously.
- **Priority scheduling**: Some processes have higher priority (e.g., system processes, real-time audio playback) and get CPU time more frequently than lower-priority processes.
- The OS handles **context switching** — saving the state (register values, program counter, memory maps) of the current process before loading the state of the next process. This has overhead — too many context switches waste CPU time.
- With multi-core CPUs, the OS can run different processes on different cores truly simultaneously.

© RAG Learning · Not for redistribution

4. Peripheral (Device) Management

The OS manages communication with all hardware devices (keyboard, mouse, printer, monitor, webcam, USB drives, etc.).

- Uses **device drivers** — specialised software that translates generic OS commands into device-specific instructions. Each hardware device needs a compatible driver. A printer driver translates a "print this document" command into the specific sequence of instructions that particular printer model understands.
- The OS provides a **standard interface** for applications to use hardware without knowing the specific details. An application calls "print()" and the OS + driver handle the rest.
- **Plug-and-play:** The OS automatically detects newly connected devices and installs appropriate drivers (either from a built-in library or by downloading them).
- The OS manages **device queues** — for example, a print queue holds print jobs in order when multiple users send documents to the same printer.
- **Interrupts:** When a device needs attention (e.g., keyboard key pressed, mouse moved, data received from network), it sends an interrupt signal to the CPU. The OS pauses the current task, handles the interrupt (e.g., processes the key press), then resumes the previous task.

5. User Management

On shared or networked systems, the OS manages multiple user accounts with different levels of access.

- Each user has a unique account with username and password (or other authentication methods like biometrics or 2FA).
- The OS assigns **access levels** that determine what each user can do:
  - **Administrator:** Full control — can install/remove software, change system settings, create/delete user accounts, access all files.
  - **Standard user:** Can run applications, access their own files, change personal settings. Cannot install system software or modify other users' files.
  - **Guest:** Very limited access — typically read-only, cannot install software or change settings. For temporary or untrusted users.
- Enforces the **principle of least privilege** — users only have the permissions they need for their role, minimising damage if their account is compromised.

© RAG Learning · Not for redistribution

- Manages login/logout sessions, audit logs (who did what and when), and account policies (password complexity, expiry, lockout after failed attempts).

6. File Management

The OS organises files in a hierarchical directory (folder) structure and provides tools for managing them.

- **Directory structure:** Files are organised in a tree of folders within folders. The root is the top level (e.g., `C:\` on Windows, `/` on Linux). This hierarchy allows millions of files to be organised logically.
- **File naming and extensions:** Extensions indicate file type and which application should open them (.docx for Word, .py for Python, .jpg for images, .mp3 for audio). The OS maintains associations between extensions and applications.
- **File operations:** Create, copy, move, rename, delete, search, sort. The OS handles these through the file manager GUI or CLI commands.
- **File permissions:** Read (view contents), Write (modify contents), Execute (run as a program). Permissions can be set per user or per group. This prevents unauthorised users from reading, modifying, or deleting files they should not have access to.
- **File system:** The OS tracks which physical disk sectors belong to which files using a file system (NTFS on Windows, ext4 on Linux, APFS on macOS). The file system also tracks free space, handles fragmentation, and provides journaling (crash recovery).
- **File allocation:** When saving a file, the OS finds free space on the disk, writes the data, and updates the file allocation table to record which sectors belong to that file. When deleting, it marks those sectors as free (the data is not immediately erased — which is why deleted files can sometimes be recovered).

> **Exam Tip:** Know all six OS functions and be able to explain each one with specific details. Memory management and process management are the most commonly examined. For memory management, explain the chain: RAM full → virtual memory used → pages swapped between RAM and disk → disk is much slower → performance drops → thrashing if excessive. For process management, explain time-slicing: each process gets a small time slice of CPU time, switching so fast the user perceives simultaneous execution.

1.5.2 Utility Software

Utility software performs specific maintenance and housekeeping tasks to keep the system running efficiently and securely. Utilities are part of systems software (not application software) because they maintain the system rather than perform user tasks. Most operating systems include basic utilities, but more advanced versions can be purchased separately.

Utility	Purpose	How It Works	Key Detail
Encryption software	Protects data confidentiality by making it unreadable without the correct key	Uses a cryptographic algorithm and a key to convert plaintext into ciphertext. The data can only be read by someone who has the correct decryption key. Can encrypt individual files, folders, or entire drives.	**Full-disk encryption** (e.g., BitLocker on Windows, FileVault on macOS) protects all data if the device is lost or stolen. The thief gets the hardware but cannot read any data without the encryption key/password. Essential for laptops and portable devices that may be lost or stolen.
Defragmentation	Reorganises fragmented files on an HDD to improve read speed	Over time, as files are created, modified, and deleted, they become stored in non-contiguous (scattered) blocks across the disk surface. The read/write head must jump between different areas to read a single file, slowing access. Defragmentation rearranges file blocks into **contiguous sequences**, reducing	**Only applies to HDDs** (which have a mechanical head that must physically move to different positions). **SSDs should NOT be defragmented** — they have no moving parts (all locations are accessed at the same speed), defragmentation provides no benefit, and the unnecessary write operations reduce the SSD's limited lifespan. SSDs use **TRIM** instead, which tells the SSD which blocks are no longer in use so they can be efficiently reused.

© RAG Learning · Not for redistribution

		head movement and improving read speed.	
Data compression	Reduces file sizes for storage and transmission	**Lossless** (ZIP, PNG, FLAC): compresses by finding patterns and encoding them more efficiently. Original data is fully recoverable. **Lossy** (JPEG, MP3, MP4): removes data humans cannot easily perceive. Smaller files but some quality loss; not reversible.	Essential for: reducing download times; fitting more data on storage devices; reducing bandwidth usage for streaming; emailing large files within attachment size limits; making web pages load faster.
Backup software	Creates copies of data for disaster recovery	**Full backup:** Copies everything. Slow to create, large storage requirement, but simple to restore (just restore the single full backup). **Incremental:** Copies only files changed since the *last backup of any type*. Fast to create, minimal storage, but restoration requires the last full backup PLUS every incremental since then (slow and complex). **Differential:** Copies files changed since the *last full backup*. Middle ground — faster than full, simpler to restore	Best practice is the **3-2-1 rule**: keep 3 copies of data, on 2 different media types, with 1 copy offsite (or in the cloud). Protects against hardware failure, ransomware, theft, fire, flood, and accidental deletion. Backups should be tested regularly to ensure they can actually be restored.

		than incremental (need full + latest differential only).	
Anti-malware	Detects, quarantines, and removes malicious software	Uses **signature detection** (comparing file hash/patterns against a database of known malware) and **heuristic/behavioural analysis** (monitoring for suspicious behaviour patterns like rapid file encryption, which could indicate ransomware, or a program trying to access the webcam without permission).	Must be kept **updated** because new malware variants are created constantly. Provides both real-time protection (scans files as they are opened/downloaded/executed) and on-demand full-system scans. Some advanced solutions use machine learning to detect previously unknown threats based on their behaviour.

Backup Types Comparison

```
Scenario: You have 100 GB of data and back up nightly. Monday = full backup.
Suppose 2 GB of data changes each day.

Full backup every night:
  Mon: 100 GB  |  Tue: 100 GB  |  Wed: 100 GB  |  Thu: 100 GB
  Total storage: 400 GB.  Restore: load any single backup. Simple but wasteful.

Incremental (changes since LAST backup):
  Mon: 100 GB (full)  |  Tue: 2 GB  |  Wed: 2 GB  |  Thu: 2 GB
  Total storage: 106 GB.  Restore Thursday: need Mon + Tue + Wed + Thu.
  Fast backups, minimal storage, but complex/slow restore.

Differential (changes since LAST FULL backup):
  Mon: 100 GB (full)  |  Tue: 2 GB  |  Wed: 4 GB  |  Thu: 6 GB
  Total storage: 112 GB.  Restore Thursday: need Mon + Thu only.
  Middle ground — reasonable storage, simpler restore than incremental.
```

© RAG Learning · Not for redistribution

Exam Tip: The OS **manages hardware** and provides a platform for applications; utility software performs specific **maintenance tasks**. These are different roles — do not confuse them. Defragmentation **only benefits HDDs** (not SSDs) — this is a very commonly examined point. If asked why you should not defragment an SSD, the answer is: SSDs have no moving parts so fragmentation does not slow them down, and the unnecessary write operations reduce the SSD's limited lifespan. For backup types, know the trade-offs: full = simple to restore but slow/large; incremental = fast/small but complex restore; differential = middle ground.

© RAG Learning · Not for redistribution